

TAREA PPS02

Pruebas de SQL Injection

Autor: Carlos España Del Pozo - CETI

Contenido

Detalles de la tarea de esta unidad	3
Apartado 1: Responde a las siguientes cuestiones	3
Respuestas 1	4
Apartado 2: Crea el entorno de pruebas.....	5
Respuestas 2	5
Apartado 3: Pruebas de vulnerabilidades	10
Respuestas 3	10
Apartado 4: Preguntas finales	13
Respuestas 4	14

Detalles de la tarea de esta unidad

Caso práctico

Julián está preocupado porque necesita crear un aplicativo web que interactuara con una base de datos y sabe que uno de los principales riesgos es la entrada de datos de usuarios ya que podrían provocar acceso a información que no debiera de la base de datos. Ha buscado información sobre vulnerabilidades de este tipo y ha decidido que debe aprender como desarrollar de forma más segura

Para ello crea el entorno de pruebas Damn Vulnerable Web Application (<https://github.com/digininja/DVWA>) donde podrá configurar distintos niveles de seguridad, ver cómo se refleja en el código y probar distintos tipos de ataque y ver de qué manera se comporta el aplicativo y sobre todo ver que recibe la base de datos.

Va a probar ataques de tipo Injection o inyección que es uno de los principales riesgos identificados en OWASP.

Apartado 1: Responde a las siguientes cuestiones

Buscar información sobre la vulnerabilidad CVE-2023-39417 y rellenar la siguiente tabla

Breve descripción	
Impacto	
Productos y versiones vulnerables	
Posibles soluciones	

Buscar información del requisito ASVS v4.0.3-5.3.4 (ASVS versión 4.0.3 capítulo 5, apartado 3, requisito 4) y rellena la siguiente tabla

Breve descripción	
Niveles a los que debe aplicarse	
Categoría CWE	

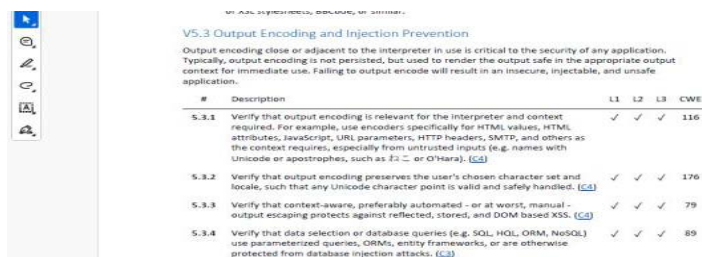
Respuestas 1

Se identifica la vulnerabilidad en la web de “CVE: Common Vulnerabilities and Exposures”:
<https://www.cve.org/>



Breve descripción	Vulnerabilidad de inyección SQL en PostgreSQL que se produce en scripts de extensiones cuando se utilizan las sustituciones @extowner@, @extschema@ o @extschema:...@ dentro de construcciones de comillas (dollar quoting, comillas simples o dobles)
Impacto	Un atacante podría modificar o borrar datos explotando la vulnerabilidad, tiene una gravedad ALTA (CVSS 7.5)
Productos y versiones vulnerables	A sistemas que usan PostgreSQL en Red Hat, como: Red Hat Enterprise Linux 8 y 9 Red Hat Software Collections Red Hat Advanced Cluster Security
Posibles soluciones	Actualizar PostgreSQL con los parches oficiales de Red Hat

Se identifica el requisito en la url:
<https://raw.githubusercontent.com/OWASP/ASVS/v4.0.3/4.0/OWASP%20Application%20Security%20Verification%20Standard%204.0.3-es.pdf>



#	Description	L1	L2	L3	CWE
5.3.1	Verify that output encoding is relevant for the interpreter and context required. For example, use encoders specifically for HTML values, HTML attributes, JavaScript, URL parameters, HTTP headers, SMTP, and others as the context requires, especially from untrusted inputs (e.g. names with Unicode or apostrophes, such as “ or O'Hara). [C-3]	✓	✓	✓	116
5.3.2	Verify that output encoding preserves the user's chosen character set and locale, such that any Unicode character point is valid and safely handled. [C-3]	✓	✓	✓	176
5.3.3	Verify that context-aware, preferably automated - or at worst, manual - output escaping protects against reflected, stored, and DOM-based XSS. [C-3]	✓	✓	✓	79
5.3.4	Verify that data selection or database queries (e.g. SQL, HQL, ORM, NoSQL) use parameterized queries, ORMs, entity frameworks, or are otherwise protected from database injection attacks. [C-3]	✓	✓	✓	89

Breve descripción	Las consultas a bases de datos deben estar protegidas contra inyecciones mediante el uso de consultas parametrizadas, ORMs u otros mecanismos seguros.
Niveles a los que debe aplicarse	L1, L2, L3
Categoría CWE	CWE-89 – SQL Injection

Apartado 2: Crea el entorno de pruebas

- Hoy en día la mayoría de los equipos de desarrollo utilizan contenedores para montar los entornos de desarrollo. En la página web del proyecto de DVWA (<https://github.com/digininja/DVWA>) explican como instalar la aplicación con Docker, que es una de las herramientas más habituales para trabajar con contenedores. Sigue las siguientes instrucciones:
 1. Instalar Docker y si es necesario docker compose (en algunas versiones ya viene con docker). En la página oficial de Docker explican cómo instalarlo
 2. Descargar el proyecto DVWA y descomprimirlo
 3. Abrir una terminal y cambiar al directorio donde se ha descomprimido.
 4. Ejecutar la línea de comandos: **docker compose up -d**
- Una vez instalado, para acceder a DVWA:
 - Abrir un navegador con la dirección <http://localhost:4280> El usuario es admin y la clave es admin
 - La primera vez que se abre la aplicación es necesario pulsar en el botón **Create/Reset Database**
 - A partir de ese momento el usuario es **admin** y la clave es **password**
 - Para cambiar el nivel de seguridad debes ir a **DVWA Security**

Respuestas 2

A continuación, detallo el proceso de instalación de Docker y descargar e instalar el proyecto DVWA

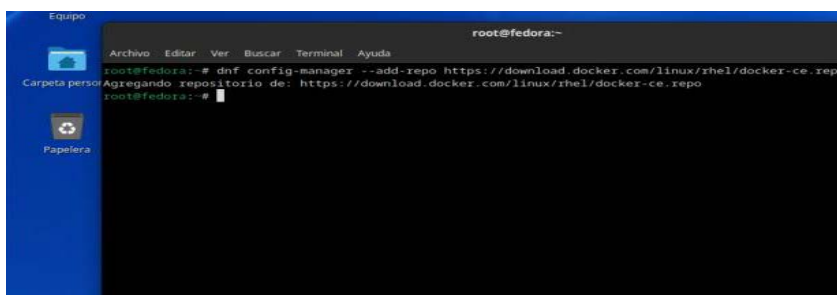
INSTALACIÓN DOCKER:

Realizamos la instalación de Docker para la versión de SO Fedora de nuestra Máquina Virtual:

La guía de referencia a utilizar: <https://docs.docker.com/engine/install/fedora/>

Instalamos el repositorio:

```
dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
```



Instalamos los paquetes necesarios:

```
dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

```

root@fedora:~# dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
El paquete docker-ce-cli-1:27.3.1-1.fc39.x86_64 ya está instalado.
El paquete docker-buildx-plugin-0.17.1-1.fc39.x86_64 ya está instalado.
El paquete docker-compose-plugin-2.29.7-1.fc39.x86_64 ya está instalado.
Dependencias resueltas.
=====
Paquete                Arquitectura  Versión      Repositorio  Tam.
-----
Instalando:
containerd.io          x86_64       1.7.23-3.1.fc39  docker-ce-stable  42 M
docker-ce              x86_64       3:27.3.1-1.fc39  docker-ce-stable  27 M
Instalando dependencias:
containerd-selinux     x86_64       2:2.222.0-1.fc39  fedora            55 k
fuse-overlayfs         x86_64       1.13-1.fc39      updates           66 k
libcgroupp             x86_64       3.0-3.fc39       fedora            74 k
libslirp              x86_64       4.7.0-4.fc39     fedora            75 k
slirp4netns            x86_64       1.2.2-1.fc39     updates           47 k
Instalando dependencias débiles:
docker-ce-rootless-extras x86_64       27.3.1-1.fc39  docker-ce-stable  4.4 M

Resumen de la transacción
-----
Instalar: 8 Paquetes
Tamaño total de la descarga: 74 M

```

Arrancamos, habilitamos y comprobamos el servicio:

```
systemctl enable --now docker.service
```

```
systemctl status docker.service
```

```

root@fedora:~# systemctl enable --now docker.service
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service - /usr/lib/systemd/system/docker.service.
root@fedora:~# systemctl status docker.service
systemd status for docker.service
* docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/service.d
            └─lib.timeout-soft.conf
   Active: active (running) since Sun 2025-12-21 13:23:17 CET; 9s ago
     TriggeredBy: * docker.socket
       Docs: https://docs.docker.com
     Main PID: 3610 (dockerd)
       Tasks: 10
      Memory: 62.0M
         CPU: 334ms
    CGroup: /system.slice/docker.service
            └─3610 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

dic 21 13:23:16 fedora dockerd[3610]: time="2025-12-21T13:23:16.832564223+01:00" level=info msg="FirewallD: interface
dic 21 13:23:17 fedora dockerd[3610]: time="2025-12-21T13:23:17.328180203+01:00" level=info msg="Default bridge (doc
dic 21 13:23:17 fedora dockerd[3610]: time="2025-12-21T13:23:17.510491235+01:00" level=info msg="FirewallD: interface
dic 21 13:23:17 fedora dockerd[3610]: time="2025-12-21T13:23:17.792376377+01:00" level=info msg="Loading containers.
dic 21 13:23:17 fedora dockerd[3610]: time="2025-12-21T13:23:17.928831924+01:00" level=warning msg="WARNING: bridge
dic 21 13:23:17 fedora dockerd[3610]: time="2025-12-21T13:23:17.921883603+01:00" level=warning msg="WARNING: bridge
dic 21 13:23:17 fedora dockerd[3610]: time="2025-12-21T13:23:17.921884485+01:00" level=info msg="Docker daemon" com
dic 21 13:23:17 fedora dockerd[3610]: time="2025-12-21T13:23:17.921279772+01:00" level=info msg="Daemon has complete
dic 21 13:23:17 fedora dockerd[3610]: time="2025-12-21T13:23:17.993765009+01:00" level=info msg="API listen on /run
dic 21 13:23:17 fedora systemd[1]: Started docker.service - Docker Application Container Engine

```

Ver la version de Docker compose integrada:

```
docker compose version
```

```

root@fedora:~# docker compose version
Docker Compose version v2.29.7
root@fedora:~#

```

Descarga del proyecto DVWA

Realizamos la descarga de DVWA desde su repositorio oficial:

<https://github.com/digininja/DVWA>

Primero de todo instalamos git:

```
yum install git
```

[illegible]

Descargamos e Instalamos el repositorio:

```
git clone https://github.com/digininja/DVWA.git
```

```

root@fedora:~# git clone https://github.com/digininja/DVWA.git
Clonando en 'DVWA'...
remote: Enumerating objects: 5622, done.
remote: Total 5622 (delta 0), reused 0 (delta 0), pack-reused 5622 (from 1)
Recibiendo objetos: 100% (5622/5622), 2.64 MiB | 8.21 MiB/s, listo.
Resolviendo deltas: 100% (2809/2809), listo.
root@fedora:~#

```

Entramos en el Directorio y revisamos los ficheros, para comprobar que existe el Docker-compose.yml

```
cd DVWA
ls -l
cat compose.yml
```

```

Archivo Editor Ver Buscar Terminal Ayuda
root@fedora:~/DVWA# cat compose.yml
volumes:
  dvwa:

Ca networks:
  dvwa:

services:
  dvwa:
    build: .
    image: ghcr.io/digininja/dvwa:latest
    # Change "always" to "build" to build from local source
    pull_policy: always
    environment:
      - DB_SERVER=db
    depends_on:
      - db
    # Uncomment the next 2 lines to serve local source
    # volumes:
    #   - ./var/www/html
  networks:
    - dvwa
  ports:
    - 127.0.0.1:4280:80
  restart: unless-stopped

db:
  image: docker.io/library/mariadb:10
  environment:
    - MYSQL_ROOT_PASSWORD=dvwa
    - MYSQL_DATABASE=dvwa
    - MYSQL_USER=dvwa
    - MYSQL_PASSWORD=p@ssw0rd
  volumes:
    - dvwa:/var/lib/mysql
  networks:
    - dvwa
  restart: unless-stopped

```


Finalmente ejecutamos el compose para arrancar los contenedores necesarios:

```
docker compose up -d
docker ps -a
```

```

root@fedora:~/DVWA# docker compose up -d

[*] Running 18/29
C:  ✓ dwva [-----] 206.9MB / 233.6MB Pulling
    ✓ 1733a4c05954 Pull complete
    ✓ c4d9a4293ba9 Pull complete
    ✓ 55359fab7d05 Extracting [-----] 26.74MB/117.8MB
    ✓ 34e5200d4a11 Download complete
    ✓ 1743a94a854 Download complete
    ✓ 5898687039c Download complete
    ✓ 3b9bbe896da Download complete
    ✓ c04db161a8fe Download complete
    ✓ 6dc0b18ebf5 Download complete
    ✓ adfa7c2a52f5 Download complete
    ✓ a434e8639380 Download complete
    ✓ 953c70bdf28e Download complete
    ✓ 9a2e74e4ebd1 Download complete
    ✓ 4f4fb700ef54 Download complete
    ✓ a7cbe13bc172 Download complete
    ✓ d4c4fa9b892c Download complete
    ✓ 6078c45098eb Download complete
    ✓ 71aa5d85f7 Download complete
    ✓ bee2ac04bc2b Downloading [-----] 7.619MB/25.87MB
    ✓ db [-----] Pulling
    ✓ 7e4dc6156bb Downloading [-----] 7.66MB/29.54MB
    ✓ 9629f0e42e43d Download complete
    ✓ 62c1a43ca7d9 Waiting
    ✓ 2540807f4e100 Waiting
    ✓ 892acd9bdcf5 Waiting
    ✓ 0aa9032ee110 Waiting
    ✓ 96d7347ba317 Waiting
    ✓ c7fd9a9765de Waiting

```

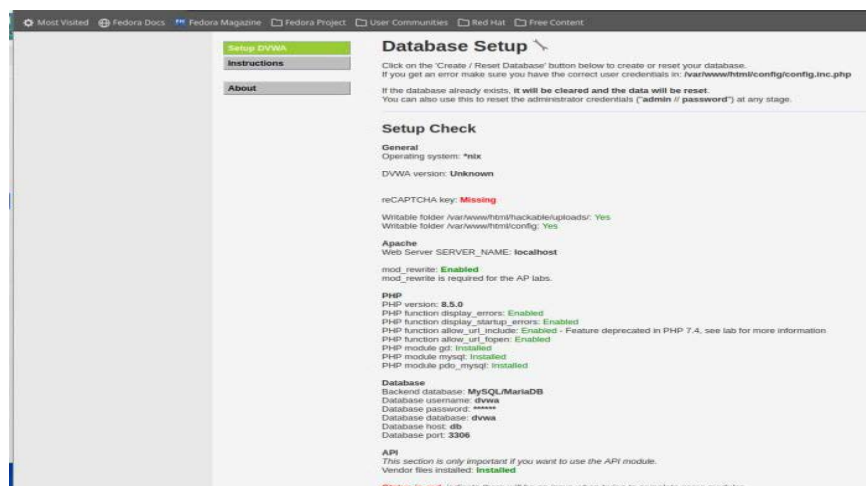
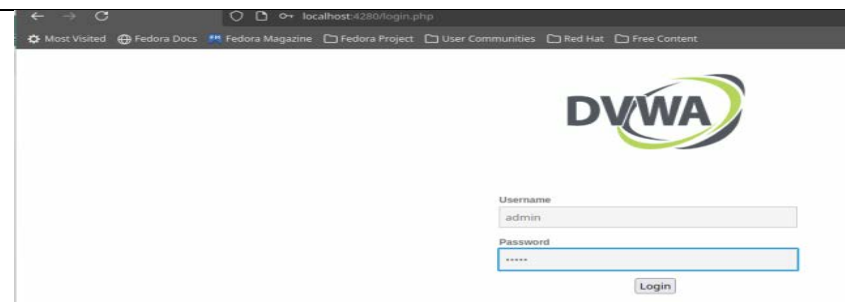
```

root@fedora:~/DVWA# docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
b327f48e4da   ghcr.io/digininja/dvwa:latest     "docker-php-entrypoi..." 13 seconds ago Up 10 seconds    127.0.0.1:4280->80/tcp   dvwa-dvwa-1
4e81229529ab   mariadb:10                         "docker-entrypoint.s..." 13 seconds ago Up 12 seconds    3306/tcp              dvwa-db-1

```

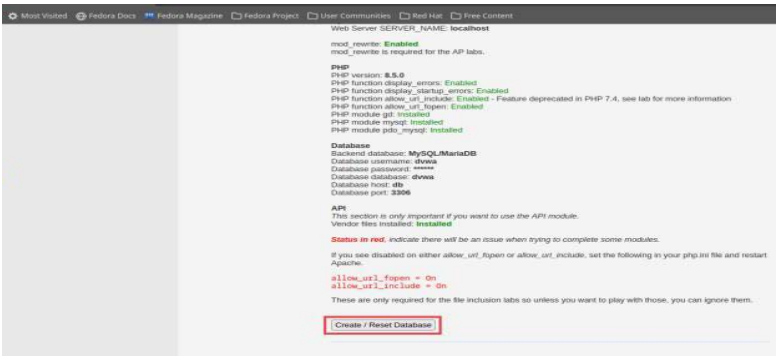
En la segunda parte accedemos al DVWA desde el navegador:

<http://localhost:4280>

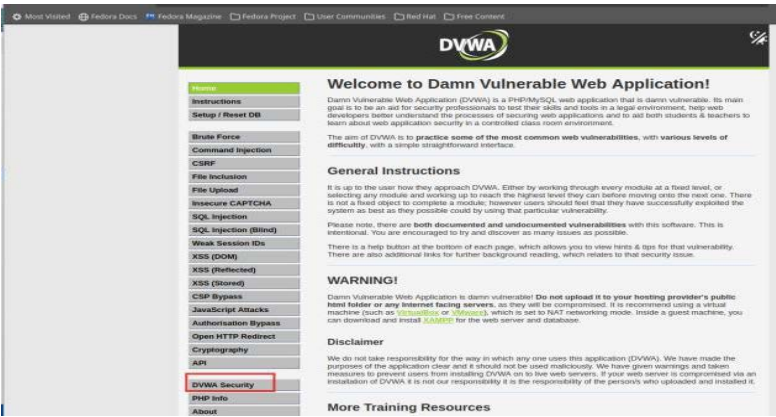


Vemos que la primera vez que se accede, DVWA nos solicita inicializar la base de datos.

Pulsamos el botón Create / Reset Database



Accedemos nuevamente para Cambiar el nivel de seguridad:



Seleccionamos el modulo deseado:



Apartado 3: Pruebas de vulnerabilidades

Procederemos a realizar distintas pruebas de SQL Injection

- Accederemos en modo **Low**
- Pulsar en el menú lateral **SQL Injection**
- Primero comprobaremos como se comporta nuestro aplicativo cuando le damos IDs de usuarios (puedes probar con 1, 2, 3..)
- Trataremos de ver qué sucede cuando le damos resultados no esperados, por ejemplo tres comillas simples '''
- Revisaremos el código del aplicativo web para entender qué ha sucedido y que consulta (query) se ha intentado realizar. Para ver el código pulsar el **botón View Source** que hay al final de la página.
- Probaremos con otros IDs como: ' OR '1'='1 (muy importante poner bien las comillas)
- Revisaremos el código del aplicativo web para entender qué ha sucedido y que consulta (query) se ha realizado. Para ver el código pulsar el **botón View Source** que hay al final de la página.
- Probaremos a cambiar el modo de seguridad de DVWA a **Impossible** (dónde no debería haber vulnerabilidades) y volveremos a realizar todos los pasos de este apartado: probar con un id de usuario, probar con tres comillas, probar con ' OR '1'='1 y revisar el código del aplicativo.

Respuestas 3

A continuación, detallo las pruebas solicitadas:

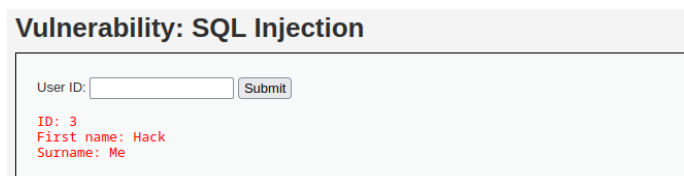
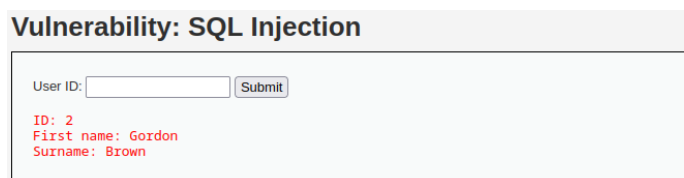
Configurar en el aplicativo DVWA el nivel de seguridad LOW:



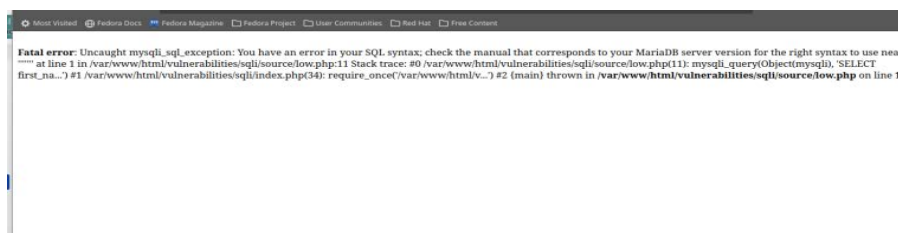
Pulsar en el menú lateral SQL Injection solicitado:



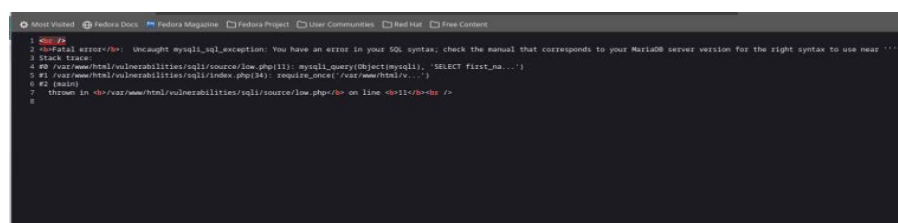
Realizamos diversas pruebas con los ids de usuario 1,2,3:



Realizamos pruebas con 3 comillas simples:



Comprobamos el código fuente de la pagina para ver que ha sucedido:



Probaremos con otros IDs como: ' OR '1'='1 para ver la vulnerabilidad ejecutada:



“

Vulnerability: SQL Injection

User ID:

' OR '1'='1

Vulnerability: SQL Injection

User ID:

Apartado 4: Preguntas finales

- **Indica cómo se ha comportado el aplicativo web:**
 - En el caso 3 comillas simples en el modo **Low**
 - En el caso ' OR '1'='1 en el modo **Low**
 - En el caso 3 comillas simples en el modo **Impossible**
 - En el caso ' OR '1'='1 en el modo **Impossible**
- **Indica qué consulta exacta (el código SQL) crees que se ha intentado/se ha lanzado en la base de datos:**
 - En el caso 3 comillas simples en el modo **Low**
 - En el caso ' OR '1'='1 en el modo **Low**
 - En el caso 3 comillas simples en el modo **Impossible**
 - En el caso ' OR '1'='1 en el modo **Impossible**
- **Realiza una comparativa del código php del modo Low y del modo Impossible ¿Qué diferencias ves?**
- **Revisa la entrada de OWASP para este tipo de vulnerabilidad (https://owasp.org/www-community/attacks/SQL_Injection). ¿Cómo crees que afecta el código php del nivel low y del nivel impossible a la vulnerabilidad que estábamos explotando?**

Respuestas 4

El aplicativo se ha comportado de la siguiente manera:

- En el caso 3 comillas simples en el modo **Low**:
El aplicativo web muestra un error de base de datos, por lo tanto, no valida correctamente.
- En el caso ' OR '1'='1 en el modo **Low**:
La aplicación devuelve todos los usuarios de la base de datos.
- En el caso 3 comillas simples en el modo **Impossible**:
La aplicación no muestra ningún error, esto indica que la entrada está controlada y no rompe la consulta.
- En el caso ' OR '1'='1 en el modo **Impossible**:
La inyección no funciona, indicando que la entrada de usuario no se interpreta como parte del código SQL.

La consulta SQL que se ha intentado lanzar, ha podido ser:

- En el caso 3 comillas simples en el modo **Low**:
Algo similar a: `SELECT first_name, last_name FROM users WHERE user_id = "";`
- En el caso ' OR '1'='1 en el modo **Low**:
Algo parecido a: `SELECT first_name, last_name FROM users WHERE user_id = " OR '1'='1';`
- En el caso 3 comillas simples en el modo **Impossible**:
Algo parecido a: `SELECT first_name, last_name FROM users WHERE user_id = :id;`
- En el caso ' OR '1'='1 en el modo **Impossible**:
Algo similar a: `SELECT first_name, last_name FROM users WHERE user_id = :id;`

La comparative de código sería:

LOW

- No existe una validación de datos.
- No se usan consultas preparadas.
- Es vulnerable a SQL Injection.

IMPOSSIBLE

- Se valida el tipo de dato.
- La entrada del usuario nunca se ejecuta como código SQL.
- No es vulnerable a SQL Injection.

Revisión de entrada en OWASP:

Según OWASP, la **inyección SQL** ocurre cuando una aplicación:

- No valida las entradas del usuario
- Construye consultas SQL dinámicas concatenando datos
- No usa consultas parametrizadas

En el nivel Low El código PHP cumple exactamente las condiciones de una SQL Injection descritas por OWASP.

En el nivel imposible el código PHP aplica las medidas necesarias recomendadas por OWASP.